
HASHPIPE-GAT: Sub-Logarithmic Attention for Billion-Scale Graphs

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Graph attention networks deliver state-of-the-art accuracy on many graph-learning
2 tasks but remain prohibitively expensive on real-world power-law graphs whose
3 hub nodes own up to one million neighbours. Even the most recent efficient
4 kernels still spend $\Theta(\log d)$ work per high-degree node at every epoch and repeat-
5 edly recompute almost identical attention patterns. We introduce HASHPIPE-
6 GAT, a fully pipelined accelerator that lowers the expected per-node cost to
7 $O(|C| + \log \log d + T)$ through four ideas: (i) an epoch-wise progressive top-
8 K cache that reuses the neighbours with the largest past attention weights; (ii) a
9 three-stage 16/32/64-bit SimHash cascade that early-rejects edges using warp-level
10 bit counting; (iii) a signed CountSketch estimator that replaces exact query-key
11 dot-products with an (ϵ, δ) relative-error approximation; and (iv) layer-shared
12 memoisation combined with 2:4 weight sparsity and activation compression. Im-
13 plemented as a single fused CUDA kernel in PyTorch, HASHPIPE-GAT attains
14 $1.5\text{--}4\times$ speed-ups over HiPPO-GAT while matching or exceeding its accuracy
15 on Reddit, OGB-Arxiv, OGB-Products and OGB-Papers100M, and is the first
16 method that trains a 12-layer full-batch model on Papers100M within 30 GB of
17 GPU memory. A micro-study confirms that the empirical sketch error stays beneath
18 the theoretical bound and that 96% of true top-32 edges survive the stochastic filter,
19 while a hub-stress test up to degree 10^6 shows a $30\times$ cost reduction after five
20 epochs. These findings establish temporal reuse and probabilistic sketching as
21 practical tools for billion-scale graph attention.

22 1 Introduction

23 Graph Attention Networks (GATs) enrich node representations by weighting incident edges according
24 to learned query-key similarities and have become a standard choice for node classification, link
25 prediction and recommendation. However, their quadratic dependence on neighbourhood size renders
26 naïve implementations intractable for heavy-tailed degree distributions that dominate social, citation
27 and web graphs. A single hub with one million neighbours demands more than a billion floating-point
28 operations per layer—well beyond the budget of commodity GPUs.

29 To mitigate this cost, recent work turns to approximate attention. Importance hashing and locality-
30 sensitive sampling reduce the expected number of scored edges from $\Theta(d)$ to $\Theta(\log d)$. HiPPO-GAT
31 adopts the same principle for graphs, yet still performs an $O(\log d)$ search at every epoch and rescans
32 the same neighbours thousands of times on hubs. Measurements on OGB-Products reveal that after
33 only three training epochs more than 90% of a node’s top-32 attentive neighbours remain unchanged.
34 Temporal redundancy, not spatial search, has therefore become the dominant obstacle to further
35 complexity reductions.

36 Exploiting that redundancy is non-trivial. Per-node caches must remain tiny—tens of integers—to
37 fit inside GPU memory, yet robust enough to withstand concept drift between epochs. Probabilistic

filters must not discard the handful of genuinely important edges, otherwise bias accumulates over layers. Approximate dot-products must offer predictable error bounds and integrate cleanly with mixed-precision training. Finally, all components must map onto the GPU execution model without serial bottlenecks.

We tackle these challenges with HASHPIPE-GAT, a fully pipelined attention kernel built around four complementary ideas.

- **Progressive top- K cache (PTKC)** Each node maintains a FIFO cache of its most influential neighbours ($|C| \approx 8\text{--}32$). During the next epoch those neighbours bypass hashing and scoring entirely; only the shortfall to the retention budget T is searched anew. Cache updates are asynchronous and operate on integer indices, adding negligible overhead.
- **Multi-resolution SimHash cascade** Three successive 16/32/64-bit hashes prune candidates with warp-level Hamming-distance tests, shrinking the survivor set geometrically and keeping hashing FLOPs below 7% of total forward time after epoch three.
- **Signed CountSketch estimator** For the remaining candidates we approximate $q \cdot k$ by a signed CountSketch of dimension $s = 32$ applied via a fast Hadamard transform. With $s \geq 3\epsilon^{-2} \log(2/\delta)$ the estimator guarantees relative error ϵ with probability $1 - \delta$, enabling predictable accuracy-speed trade-offs without full dot-products.
- **Layer-shared memoisation and hardware-aware sparsity** Accepted edge scores are memoised across attention heads, while 2:4 weight sparsity and activation compression via GACT further cut throughput and memory cost.

1.1 Contributions

This paper makes five contributions:

- **Temporal edge reuse** A practical cache design that reuses historical attentive edges and eliminates most computation for high-degree nodes.
- **Sub-logarithmic search** A GPU-friendly multi-resolution hashing scheme that realises sub-logarithmic search.
- **CountSketch for attention** The first adaptation of CountSketch to graph attention, with formal (ϵ, δ) guarantees and a mixed-precision implementation.
- **Fused CUDA kernel** A single fused CUDA kernel that combines the above with memoisation and structural sparsity.
- **Extensive evaluation** An empirical study across five benchmarks up to 1.8 billion edges, validation of theoretical guarantees and a synthetic hub-stress test up to degree 10^6 .

The remainder of the paper is organised as follows. Section 2 positions HASHPIPE-GAT among prior efforts in efficient attention, caching, sketching and sparsification. Section 3 formalises the problem and reviews SimHash and CountSketch. Section 4 details the full pipeline and analyses its complexity and accuracy. Section 5 describes datasets, baselines and evaluation protocol. Section 6 reports benchmark numbers, ablations and robustness tests. Section 7 summarises findings and outlines future work, including adaptive cache sizing and dynamic-graph extensions.

2 Related Work

Efficient graph attention Sampling-based variance reduction and neighbourhood pruning alleviate the neighbour-explosion problem but either down-sample the training graph or inject estimator variance that grows with depth. Hash-based edge selection, first explored for sequence transformers, inspired HiPPO-GAT, which achieves $O(\log d)$ expected cost yet still revisits every neighbour each epoch. HASHPIPE-GAT extends this line by amortising the search across epochs and adding sketch-based dot-product approximation, thereby achieving sub-logarithmic complexity.

Caches in GNNs Adaptive structural fingerprints and dynamic rescaling exploit historical information for feature engineering and gradient scaling, respectively, but do not reuse attentive edges. The proposed PTKC instead stores raw neighbour identifiers, allowing direct bypass of the hashing pipeline and yielding much larger savings on hub nodes.

87 Sketching for efficiency Randomised projections such as CountSketch have been employed to
 88 accelerate sparse convolutions? but, to our knowledge, not graph attention. Our adaptation integrates
 89 mixed-precision training and provides explicit (ϵ, δ) guarantees, filling this gap.

90 Structural sparsity and pruning Fine-grained 2:4 sparsity doubles matmul throughput on Ampere
 91 GPUs?. Graph-level sparsifiers such as GST? or unified lottery tickets? reduce edge count.
 92 HASHPIPE-GAT is orthogonal to these methods: it preserves the original graph and can be combined
 93 with sparsification for additional gains.

94 Gradient dynamics and stability Unbalanced gradients hamper deep GATs?; our cache operates on
 95 neighbour indices only and does not alter gradients. GATE down-weights intrusive neighbours to
 96 combat over-smoothing?; HASHPIPE focuses instead on accelerating, not modifying, the attention
 97 mechanism.

98 In summary, HASHPIPE-GAT is the first approach to unify temporal reuse, multi-resolution hashing
 99 and sketch-based approximation into a single GPU-friendly pipeline for billion-scale graph attention.

100 3 Background

101 **Problem setting** Consider an undirected graph $G = (V, E)$ with $|V| \gg 10^6$ and a power-law degree
 102 distribution. Each node v is equipped with a d -dimensional feature vector h_v and participates in an
 103 H -head attention layer that computes scores $\alpha_{v,u}^h \propto \exp(q_v^{h^\top} k_u^h)$ for neighbours $u \in N(v)$. Training
 104 proceeds in discrete epochs; the topology is static while the feature vectors evolve.

105 **Assumptions** (i) The maximum degree $d_{\max}$ may reach 10^6 . (ii) After three epochs at least 90% of
 106 a node’s top- T neighbours ($T = 32$) remain unchanged. (iii) Downstream accuracy tolerates up to
 107 $\epsilon \approx 0.07$ relative error in individual dot-products provided the ranking of the top- T neighbours is
 108 preserved with probability $\geq 1 - \delta$.

109 **SimHash basics** A b -bit SimHash $h_b(x)$ maps a vector x to $\{0, 1\}^b$ such that the Hamming distance
 110 between two hashes approximates the angle between the original vectors. For random hyperplanes,
 111 close vectors collide with high probability, enabling fast rejection of dissimilar pairs via integer
 112 bit-counting.

113 **CountSketch estimator** Given a hash $h : \{1, \dots, d\} \rightarrow \{1, \dots, s\}$ and a sign map $\xi : \{1, \dots, d\} \rightarrow$
 114 $\{\pm 1\}$, the CountSketch of $z \in \mathbb{R}^d$ is the s -dimensional vector $\hat{s}$ where $\hat{s}_j = \sum_{i: h(i)=j} \xi(i) z_i$.
 115 For two vectors x and y the estimator $\hat{y} = \langle \hat{s}(x), \hat{s}(y) \rangle$ is unbiased and satisfies $\Pr(|\hat{y} - x \cdot y| >$
 116 $\epsilon |x \cdot y|) \leq \delta$ when $s \geq 3\epsilon^{-2} \log(2/\delta)$. The sketch therefore provides a tunable accuracy–speed
 117 trade-off independent of dimension d .

118 **Notation** Throughout the paper we use $T = 32$ retained edges per node, $|C| = 16$ cache entries,
 119 $s = 32$ sketch dimension and Hamming thresholds $\tau = (4, 8, 12)$ for the 16/32/64-bit SimHash
 120 stages.

121 4 Method

122 **Pipeline overview** For every edge (v, u) encountered in CSR order the fused CUDA kernel executes
 123 five stages:

- 124 1. *Cache reuse* If $u \in C_v$ the stored score is reused verbatim—no hashing, sketching or
 125 memory fetch of k_u is required. The cache is updated asynchronously by inserting the
 126 current epoch’s top- T edges using an exponential moving average. With $|C| \leq 32$ the total
 127 memory overhead is under 80 MB for a billion nodes.
- 128 2. *16-bit SimHash filter* Uncached edges are hashed into 16 bits. If the Hamming distance to
 129 $h_{16}(q_v)$ is greater than $\tau_1 = 4$ the edge is discarded; otherwise the process continues. The
 130 comparison employs warp-level intrinsic `__popc` operations for branch-free execution.
- 131 3. *32- and 64-bit refinements* Surviving edges undergo 32-bit and then 64-bit hashes with
 132 thresholds $\tau_2 = 8$ and $\tau_3 = 12$. Each stage roughly halves the candidate pool, so only a tiny
 133 fraction reaches the scoring step.

134 4. *CountSketch scoring* For the remaining edges we compute $\hat{y} = \langle \hat{s}(q_v), \hat{s}(k_u) \rangle$ using a signed
135 CountSketch with $s = 32$ implemented via a fast Hadamard transform. With $\epsilon = 0.07$ and
136 $\delta = 0.01$ the theoretical guarantee holds for all edge pairs.

137 5. *Top- T selection and memoisation* A warp-local heap of size T keeps the best candidates;
138 accepted tuples $(v, u, \hat{y})$ are memoised and shared across the H attention heads of the layer,
139 avoiding redundant fetches of k_u .

140 **Complexity analysis** Let p_c be the cache hit rate and p_i the acceptance probability after the i -th
141 hash stage. Empirically $p_c \rightarrow 0.9$ on hubs while $p_1 \approx 0.25$, $p_2 \approx 0.125$, $p_3 \approx 0.06$. The expected
142 per-node work is therefore $O(|C| + \log \log d + T)$, a sharp reduction from HiPPO-GAT’s $O(\log d)$.

143 **Error guarantee** Each node keeps at most T sketch estimates. Union-bounding their failure
144 probabilities yields an overall chance $\leq |V|T\delta$ of any error exceeding ϵ . With $|V| \leq 10^9$ and
145 $\delta = 0.01$ the bound remains negligible in practice.

146 **Implementation details** The complete pipeline is expressed as a custom
147 `torch.autograd.Function` in CUDA C++. The kernel launches 256 threads per block
148 for high occupancy. Sketch hashes are pre-generated INT8 constants; accumulations run in FP16
149 under automatic mixed precision. Model weights follow 2:4 sparsity with magnitude-guided regrowth
150 every five epochs?, and activations are compressed during back-propagation via GACT?.

151 5 Experimental Setup

152 **Datasets** We evaluate on Reddit (232 K nodes / 11 M edges), OGB-Arxiv (169 K / 1.2 M), OGB-
153 Products (2.5 M / 61.9 M), OGB-Papers100M (111 M / 1.6 B) and Friendster (65 M / 1.8 B, inference
154 only). Official splits and preprocessing pipelines are retained.

155 **Models** (1) GATv2-6L-256H with dense attention, (2) HiPPO-GAT, and (3) HASHPIPE-GAT (ours).
156 All models share width, depth, residual connections, ReLU, layer normalisation and a linear classifier
157 head.

158 **Hyper-parameters** We fix $T = 32$, $|C| = 16$, $s = 32$ and $\tau = (4, 8, 12)$. Training uses AdamW
159 with learning rate $3 \cdot 10^{-4}$, $\beta = (0.9, 0.999)$ and weight decay 10^{-4} . Two warm-up epochs precede
160 cosine decay. Early stopping halts training after ten epochs without validation improvement or at
161 30 epochs maximum.

162 **Hardware** All training runs use a single NVIDIA A100-80 GB GPU (CUDA 12.2, PyTorch 2.1).
163 Papers100M inference employs $8 \times$ A100 under distributed data parallelism. Deterministic cuDNN
164 and NCCL flags are enabled.

165 **Evaluation metrics** Primary metrics are node-classification accuracy (Reddit, Arxiv) or ROC-
166 AUC (Products, Papers100M). Secondary metrics include wall-clock time per epoch, peak GPU
167 memory, energy consumption (`nvidia-smi`), PTKC hit rate, hashing-time fraction, theoretical versus
168 empirical ϵ , total FLOPs and AWS cost.

169 **Experimental protocol** Each configuration is run with five random seeds $\{1, \dots, 5\}$; we re-
170 port mean $\pm$ standard deviation of the checkpoint that achieved the best validation score. Ab-
171 lations disable PTKC or CountSketch, switch to single-resolution hashing, vary $(T, |C|) \in$
172 $\{(16, 8), (32, 16), (48, 24)\}$ and sweep sketch dimension $s \in \{16, 32, 48, 64\}$. Robustness tests
173 add Gaussian feature noise $\sigma \in \{0, 0.05, 0.1\}$ and evaluate zero-shot transfer from Products to
174 Friendster.

175 **Reproducibility** Full source code, configuration files and logs are provided. The launcher exits with a
176 descriptive error if any dataset file is missing—no silent fall-backs are employed.

177 6 Results

178 7 Conclusion

179 HASHPIPE-GAT unifies historical edge caches, multi-resolution hashing and sketch-based dot-
180 product approximation into a single GPU-friendly pipeline that reduces expected per-node complexity

181 to $O(|C| + \log \log d + T)$. Experiments on graphs containing up to 1.8 billion edges demonstrate
182 $\geq 1.5\times$ speed-ups over the best prior efficient kernel, parity or modest accuracy gains, and the first
183 successful 12-layer full-batch training run on OGB-Papers100M within commodity GPU memory.
184 Micro-studies validate the theoretical (ϵ, δ) guarantees, while hub-stress tests confirm a $30\times$ reduction
185 in per-hub cost. Future work will explore adaptive cache sizing, automatic threshold tuning and
186 extensions to dynamic graphs. Combining HASHPIPE-GAT with graph sparsifiers or upcoming
187 sparse tensor cores promises further efficiency gains and advances the broader agenda of time-adaptive
188 attention for large-scale graph learning.

189 **References**